What Is Data

Duke OLLI Digital Explorers SIG

David Shamlin, July 2026

Learning Objectives

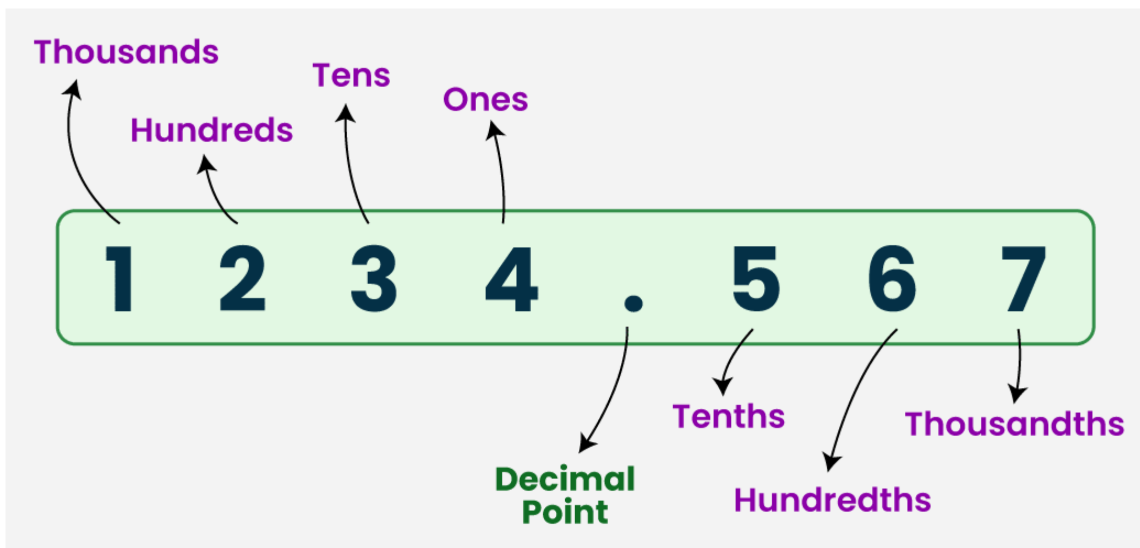
- Knowledge
  - How information is represented as digital data (a.k.a., binary data)
  - What the units and types of binary data are
  - How data is organized: common data structures
  - Data in memory vs on disk (i.e., data-in-use vs data-at-rest)
- Skills: None

Information vs Data

- Computers are information processing devices. Computers represent information to us in three primary formats: text, images (photos and videos), and sound.
- All of these forms of information are represented “inside” a computer as *digital data*, more formally referred to as *binary data*.
- Merriam-Webster defines the word *binary* as “something made of two things or parts” and (in the field of mathematics) as a “number system based only on the numerals 0 and 1” (a.k.a. “a binary number system”).
- **Key Points:**
  - In the digital world, everything is data.
  - While a piece of data can always be interpreted as an integer number, a piece of data may represent something other than an integer number (e.g., a character of the alphabet, a date, a processor operation code) depending on the context in which it is being used at the moment.

What are decimal numbers?

- When people count things, they use *decimal* numbers. The decimal numbering system has ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. When we are counting more than nine things, we use multiple digits and each digits value depends on its position in the number. The image below illustrates this with the number 1234.567:



What are binary numbers?

- In the binary number system, the digits “0” and “1” represent the values zero and one (in the same way they do in the decimal number system). To represent values greater than 1 in the binary number system, the digits “0” and “1” are combined in a similar way to the decimal digits “0” through “9”.
- The table on the left shows how the integer values zero through ten are represented as decimal and binary numbers.
- Per Britannica, computers use the binary number system binary numbers because of the “compact and reliable manner in which 0s and 1s can be represented in electromechanical devices [e.g., microchips] with two states —such as “on-off”, “open-closed,” or “go-no go”.

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

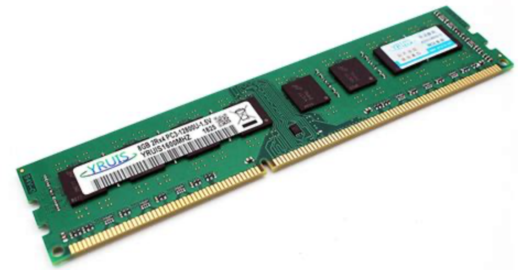
Bits: the smallest unit of digital data

- The binary digits zero and one are easily represented with electronics using a type of component called a *flip-flop* or a *latch*.
- These components work like a light switch. When a light switch is on the “on” position, electricity flows through the wires connected to the switch and the light bulb(s) connected to it causing the light bulb to glow. When a light switch is in the “off” position, electricity does not flow through the wires connected to the switch and the associated light bulb(s) do not glow.
- Similarly, when a flip-flop is closed, electricity flows through it; when a flip-flop is open, electricity does not flow through it.
- Given flip-flop have two possible states (open and closed), they can be used to represent the binary digits zero (0) when the flip-flop is open and one (1) when the switch is closed.
- A single flip-flop used in this way is called a **bit**. A bit is the smallest unit of data in the digital world.

What is a byte?

- Sets of bits are used to represent numbers greater than one (1). Eight bits used as a group is called a **byte.**
- One byte can “hold” 256 uniquely different values. I.e., eight flip-flop each with two possible states can have (2 x 2 x 2 x 2 x 2 x 2 x 2 x 2) = 2^8 = 256 unique states in total. This means that one byte can be used to represent the non-negative integer values from 0 to 255.
- When integers greater than 255 are needed, multiple bytes are used together.
    - Two bytes have 16 bits and can have 65,536 (i.e., 2^{16}) values.
    - Four bytes have 32 bits and can have 4,294,967,296 (i.e., 2^{32}) values.
- A byte can be used to represent both negative and positive integers. One of the bits is used to indicate whether the represented number is positive or negative—leaving the seven remaining bits to represent the number. When bytes are used to represent negative and positive values.
- **Key point:** A byte is the smallest unit of digital data. The value of a byte of digital data is a number—more specifically an integer value expressed as a binary number. Sometimes the value of a byte is used as an integer number; sometimes it is a code for something else (like a character from an alphabet.) The software that is using the byte of data “decides” whether the byte is treated as an integer number of a code for something else.

Bytes in Memory



- Recall from our session on hardware that memory—more formally called *random access memory* (RAM)—is a microchip that resides inside a computer. The image in the upper right hand corner is a “stick” of RAM. If we had a magic microscope that allowed us to see what is on this microchip, we would see a bunch of flip-flops.

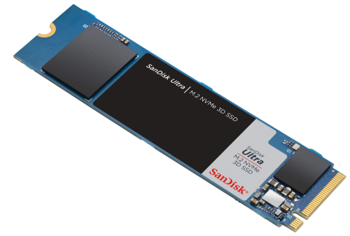
- How many flip-flops would we see? If we assume the RAM stick is 8GB of memory, we would see 68,719,476,096 flip-flops (i.e., 68.7+ billion flip-flops).

- These 68.7 billion flip-flops are arranged on the chip in a what is called an **array** which is just a software engineering term for a list of things that can be accessed by their ordinal position. E.g., the 1st thing, the 2nd thing, the 3rd thing…

- However and because individual bits only store two values, a RAM stick’s array is segmented by bytes. I.e., when accessing the n^{th} “thing” on a chip of RAM, I am access the n^{th} byte.

- So instead of saying an 8GB RAM stick has 68.7+ billion bits, we normally say an 8GB RAM stick has 8.6+ billion bytes. And that’s still a mouthful which is why we say 8 gigabytes—or 8GB. (“Giga-” is a unit prefix in the metric system that means “billion”).

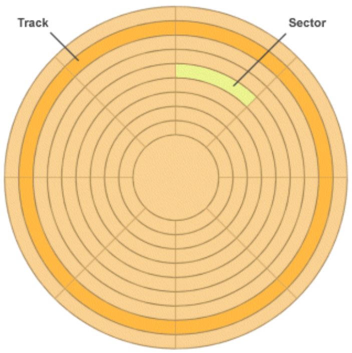
Bytes in Storage



- Recall from our session on hardware that the term “storage” is a synonym for “drive” and there are two kinds of drives: solid state drives (SDDs) and hard disk drives (HDDs) The images in the upper right hand corner are of an SDD drive and an HDD drive.

- Both of these devices work in a conceptually similar way to RAM sticks. I.e., they act as arrays of bits/bytes. The number of the bits/bytes array a drive has which is typically measured in gigabytes or terabytes (TB; a trillion bytes).

- Mechanically, an SDD is very similar to a RAM stick in that the microchip on an SDD houses a bunch of flip-flops. A key difference between the microchip on an SDD and the microchip on a RAM stick is how it behaves when the computer that contains them is powered off. When a computer is powered off, the flip-flops on a RAM stick all return to their “off” states while the flip-flops on an SSD that are in the “on” state <u>retain</u> their “on” state. The former behavior is called “volatile” and the latter behavior is called “non-volatile”.



- An HDD is mechanically different. The platter (the silver disk in the image of an HDD above) is a magnetic surface (similar to a cassette tape). The platter’s surface is segmented into tracks and sectors. The arm (which can move back and forth over the platter) has an electromagnet on its tip. When data is written on to the platter, the electromagnet creates either a positive (“north”) magnetic charge or a negative (“south”) magnetic charge at the “spot” beneath the tip of the arm; the platter will retain that charge until such time as the tip of the arm returns to that same spot and creates an opposite charge. A charged “spot” on the platter is a bit. (So eight “spots” are a byte.) The spots are organized into tracks and sectors (see the image to the left); a platter has *m* number of tracks and each track has *n* number of sectors. Each sector has *p* number of “spots” and each sector has enough “spots” for a sector to “hold” either 256 or 512 bytes. The spots in a sector are organized as arrays.

- Whereas a byte on a RAM or SSD microchip is accessed—or **addressed**—by it’s element number in the chip’s array, a byte on an HDD platter is addressed using a track number and a sector number and an array number.

- **<u>Key point</u>:** While their are differences between the mechanics of RAM, SDDs, and HDDs, they conceptually function in nearly the same way: they store data in bytes that are organized as arrays. The primary difference between these three components is that SDDs and HDDs <u>retain</u> data that are “holding” when the computer turned off while RAM looses (or “forgets”) the data it is “holding” when the computer is turned off.

How are alphabetic characters represented in bytes?

- There are 26 letters in the English alphabet and they have a well established order: a, b, c, d, e, …, z. If we assign each letter a number, we can easily use a single byte to represent the letters of the alphabet: a = 0, b = 1, c = 3, d = 4, etc.

- Using this scheme, the word “dog” can be represented as “4, 14, 6” and the word “cat” as “2, 0, 20”.

- In this scheme the sequence of numbers (0, 1, 2, …, 25) matches the way we order letters alphabetically (a, b, c, d, e, …, z). This can be used to determine whether the word “dog” comes before or after “cat” in alphabetical order:

  - “cat” = “2, 0, 20”

  - “dog” = “4, 14, 16”

- If we compare the pair of numbers used to represent the first letter of each word (i.e., 2 and 4), the greater number indicates which word follows the other in alphabetical order. I.e., 4 > 2 so “dog” comes <u>after</u> “cat” in alphabetical order.

Basic data types

As previous pages have indicated, bytes are used to represent different **types** of information. The basic of digital data types are

- Boolean (i.e., true/false values)
- Numeric
    - Integer
    - Decimal
    - Floating point
- Character

  While the example on the previous page used lowercase letters of the alphabet, the character digital data type is also used to present uppercase letters, punctuation marks, numerical digits, etc. Between two and four bytes are used to represent characters; exactly how many bytes are used to represent a character depends on the number of characters in the human language used. E.g., there are more than 50,000 different Chinese characters.
- String (i.e., an ordered set or list of characters)
- Date/time

  Dates and times are represented using integer numbers. A time value is represented as a number of nanoseconds past 12:00 AM. There are several schemes for representing dates; the most common scheme involves using the integer data type to represent the number of days from a "origin date." A positive integer value is used to represent a number of days after the origin date and a negative integer value represents a number of days before the origin date.

Common Data Structures

- The basic data types mentioned on the previous page are commonly used together to represent more complex forms of information. Think of the Contacts app on your computer; for each person you have added to your Contacts app, there are multiple pieces of information (e.g., name, email address, phone number, etc.) Each of these pieces of information are represented using a basic data type; the individual pieces of data are combined in memory into a more complex data type.

- When basic data types are used together to represent more complex information, the way the underlying data is organized is referred to as a **data structure.** A data structure is a template that describes how individual pieces of data are to be arranged in memory or storage; when actual data exists in memory or on disk arranged in the form defined by a data structure, that data is referred to as an **instance** of the data structure. Instances of a data structure are also sometimes called **objects**.

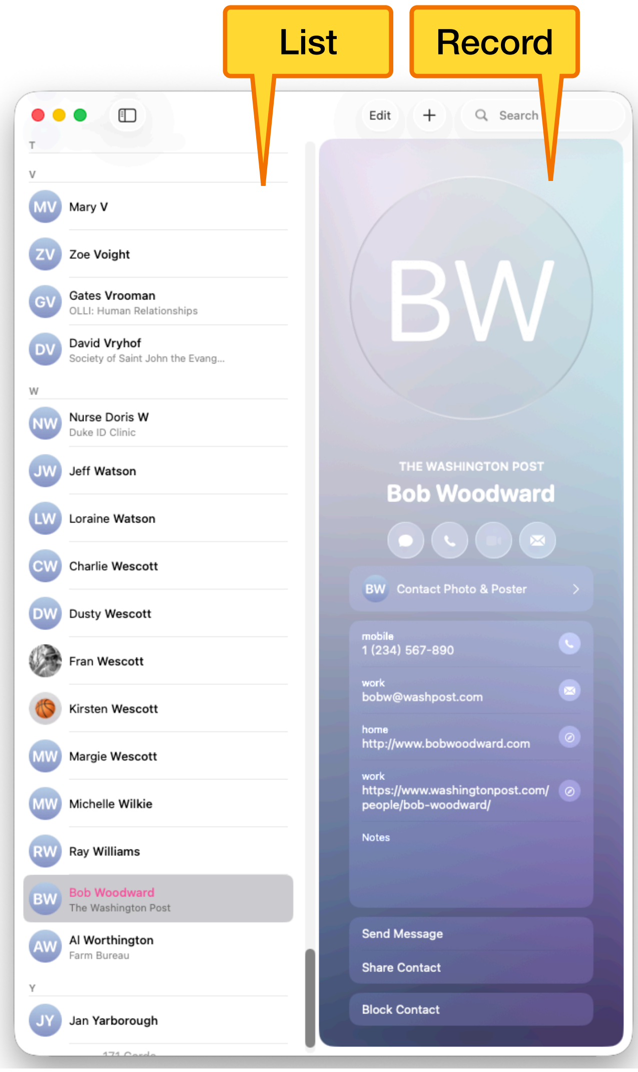
- The following pages describe several types of common (or frequently used) data structures and give examples of those types of data structures in apps.

Common Data Structures:
Records and Lists

- The image on the right shows my Contacts app with the entry for my friend Bob Woodward selected.

- The pane on the right shows the information I have added to the entry for Bob Woodward. Each individual piece of information I have entered (his name, phone number, email address, and websites) is represented in memory and on disk using the basic string data type. These pieces of information reside in memory and storage together. This is called a **record**.

- The pane on the left lists all of the people I have created entries for in my Contacts app. The data structure that represents a list of things is called a **list**.



Common Data Structures: Tables

- The image on the right (which appears on a previous page) is an example of a data structure called a **table**. This data structure organizes data in rows and columns.
- A **row** of a table is an instance of a **record**.
- A **column** of a table is an instance of a **list**.
- Table data structures are used extensively by spreadsheet apps like Excel and Numbers. **[DEMO]**

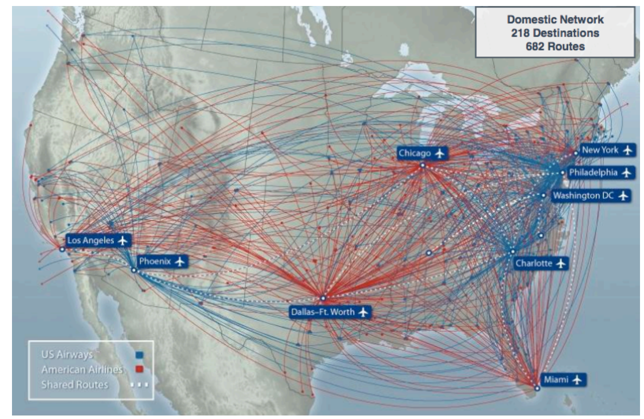
Column

Row

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

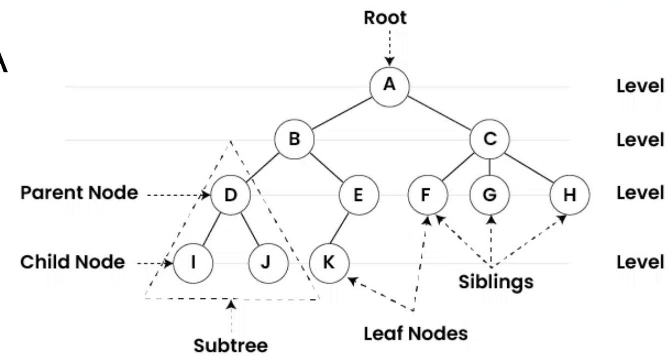
Common Data Structures: Graphs (or Networks)

- Sometimes useful to represent relationships between pieces of information. E.g., the image on the right shows an airlines U.S. flight network. Each dot represents an airport and each red/blue arc indicates flights the airline runs between airports.
- **Graph** data structures are used when relationships between pieces/units of data need to be represented; this kind of data structure is sometimes also called a **network**. Each piece/unit of data in a graph instance is called a **node**; each relationship connecting two nodes is called an **edge**. There is an entire sub-field of mathematics dedicated to graphs (formally called “graph theory”).
- The roads and highways that connect the cities and towns across the country are a network. The GPS systems in our cars represent these cities, town, roads, and highways using graph data structures. Nodes are used to represent cities and towns; edges are used to represent the roads and highways. When I enter the address of a place I want to go into my car’s GPS, the GPS often presents me with more than one route I can take to get to my desired destination and it will chose to lead my on the quickest/shortest route unless I tell it to do otherwise. Mathematicians that study/research graph theory have developed methodologies to find all the possible paths between two nodes in a graph and also to the most optimal path (e.g., shortest or quickest). The software that powers the GPS systems in our cars uses these graph methods to show us the various routes we can take to a destination and identify the fastest/shortest path.
- A quintessential example of a graph in the digital world is the Internet itself! When I send a piece of email to you, that email travels from my device to yours across the internet. As that email travels across the Internet to reach you, it goes from my computer to my ISPs data center (and probably some other data centers) to reach the site of your mail service, and it is then shipped from the mail service you use through your ISP’s data center to your computer. Similar to the GPS example above, there are multiple pathways across the Internet my email could take to reach your computer. Software that is part of the operating systems on our computers uses graph data structures to represent the Internet (with sites as nodes and connections between sites as edges) to figure out the available pathways that can be used to route my email to you.



Common Data Structures: Trees

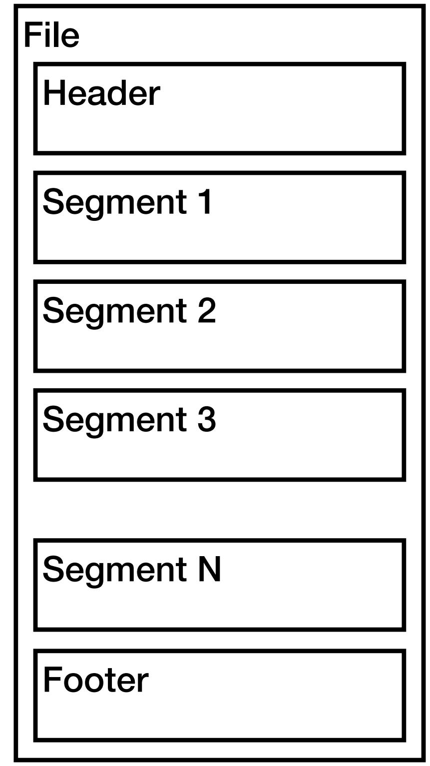
- Information is sometimes hierarchical in nature. E.g.,
  - Retailers often organize their catalog of products into departments. A department's product are then further grouped by some type. E.g., the men's clothing department might be sub-grouped into "casual wear" and "business/formal wear"; each of these sub-groups might be further organized into shirts, pants, underwear, socks, etc.
  - Apple's Menu Bar organizes menus and commands in a hierarchy.
  - Folders you create and files you save to your device's drive (i.e., storage) are arranged hierarchically.
- A type of data structure called a **tree** is used to store information that has a hierarchical nature. A tree is a specialized kind of graph.
- The node at the top of the hierarchy is called the **root** node and the nodes at the bottom of the hierarchy are called **leaf** nodes.



Common Data Structures: Files

- A **file** is a container for data stored on a drive. The data originated in memory organized using one or more of the data types and used structures described on previous pages. When the data is saved (i.e., transferred) from memory to disk, the data types and data structures used to represent the data in memory are replicated in the data that is inside the file on disk.

- The image to the right depicts how data as it resides in a file is often organized. Inside a file are a series of parts. The first part of a file is called the header; the information in the header describes how the data saved to this file from memory is organized. The data saved from memory is placed in the segments that follow the header. The segments in a file are all the same size; if the size of the data saved from memory is greater than the size of a segment, the data is divided into “chunks” that are the size of the file’s segments. After the last segment part in a file is a final part called the footer. The footer contains data that represents the file’s metadata (e.g., the name of the file, the type of the file, when the file was created and last modified, etc.)



Reference: KB, MB, GB, and TB

The acronyms KB, MB, GB, and TB are acronyms for the terms for the measurement of the size of memory and storage. The memory and storage in a computer each have a fixed size—which is an indicator of the maximum amount of data a RAM stick or disk drive can hold. The table below outlines the number of bytes defined by each of these acronyms.

		Number of bytes			
KB	kilobyte	1,024	2^{10}		“kilo-” is the metric system prefix for “thousand”
MB	megabyte	1,048,576	2^{20}	1,024KB	“mega-“ is the metric prefix for “million”
GB	gigabyte	1,073,741,824	2^{30}	1,024MB	“giga-“ is the metric prefix for “billion”
TB	terabyte	1,009,551,627,776	2^{30}	1,024GB	“tera-“ is the metric prefix for “trillion”